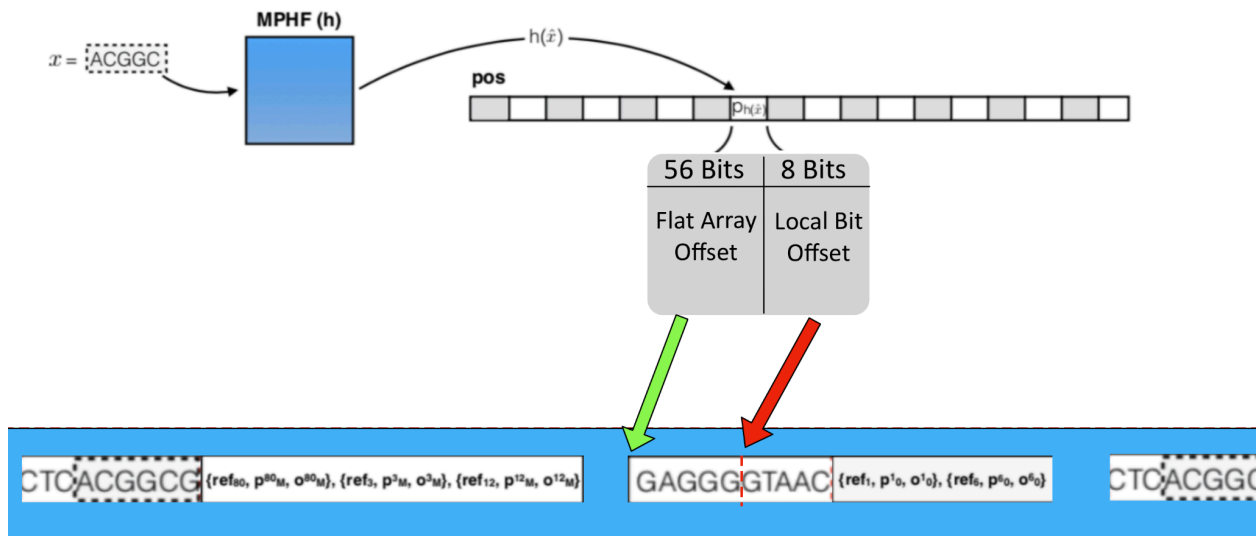


Team Members: Will Russo, Nick Egorov

The PuffAligner algorithm provides significant memory improvements over previous solutions, however its lookup operation is bottlenecked by the latency gap between the CPU and main memory. When a sequence is queried, the current Pufferfish index forces the computer to pull data from several large, disjoint tables (POS, USEQ, BV, and UTAB). This fragmented memory layout creates a system where a single K-mer query requires multiple fetches into RAM, resulting in costly cache misses.

Our group seeks to improve these cache misses by exploiting spatial locality of data. Due to the fact that a CPU requests loads a 64 byte cache line, we propose restructuring the index into a continuous Flat Array made of these 64-byte blocks. By interleaving data that is used together (unitig DNA sequence, UTAB, and ETAB), all into one 64-byte block, we make sure that fetching this sequence will also pre-load all the necessary metadata into the cache, effectively turning multiple ram fetches into one “free” cache load.

In order to enable this functionality we must modify several parts of the Pufferfish architecture. Firstly, the POS table will maintain the MPHF for lookup purposes, but each table entry will now be a bit-packed 64-bit integer. The first 56 bits will store the global offset, pointing to the start of the 64-byte block in the Flat Array. The last 8 bits store the local offset of the k-mer within that unitig string. Since our POS table directly routes the search to the unitig blocks, we completely bypass the global Bit-Vector and its operations.



During the construction of the index, long unitigs are sequentially appended in the Flat Array to guarantee memory adjacency, allowing cache prefetching to quickly stream contiguous blocks. The edge table (ETAB) within the Flat Array block would therefore be used to handle branching in the de Bruijn graph.

Goal and Accomplished work

The goal for this project is to create a prototype flat index using the aforementioned format, query it with our bit-packed POS table and benchmark it against the original Pufferfish implementation. We would specifically like to measure the reduction of cache misses and its effect on query throughput. We would also like to measure the built-time performance penalty of this approach.

So far we have done sample query collection as well as memory usage and runtime predictions.

Relevant literature and next steps:

<https://erikdemaine.org/papers/BRICS2002/paper.pdf>

<https://academic.oup.com/bioinformatics/article/34/13/i169/5045749>

<https://academic.oup.com/bioinformatics/article/33/24/4024/2725383>

<https://iswsa.acm.org/mpfh/tutorial.html>

Our next steps for the project will be to start implementing the construction algorithm necessary to build a working Flat Array, while handling certain “overflow” scenarios.

Most relevant question:

Does the increased time construction time of the interleaved flat array outweigh the performance gains during query time?